Random Forests & Ensembles

Machine Learning with Python — Lecture 1, Session 4 (Oct 19)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **Ensembles** — many weak learners make one strong learner
- **Bagging** — bootstrap samples + random feature subsets
- **Random forests** — decorrelated trees that vote
- **Boosting** — the sequential alternative (and XGBoost)
- **Feature importance** — what did the forest learn?
- **Applications** — why trees dominate tabular data

Session 4 of 4 today — thanks for a great day!

Part 1 — Random forests and ensembles

- We know single decision trees can **overfit**.
- We could choose to **underfit** single decision trees (shallow stumps)...
- ...and **combine many of them into an ensemble**.

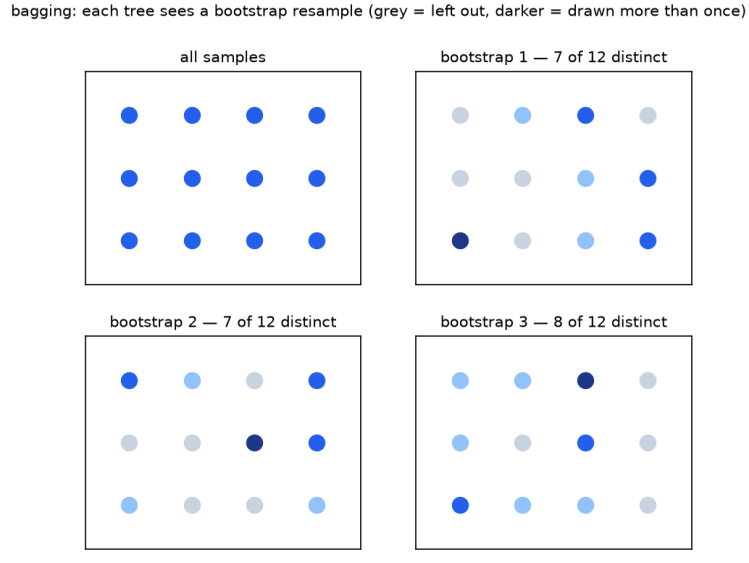
A bunch of **weak learners** → one **strong learner**. This is a surprisingly effective idea!

Random forests — bagging

Random forest = many decorrelated trees that **vote**:

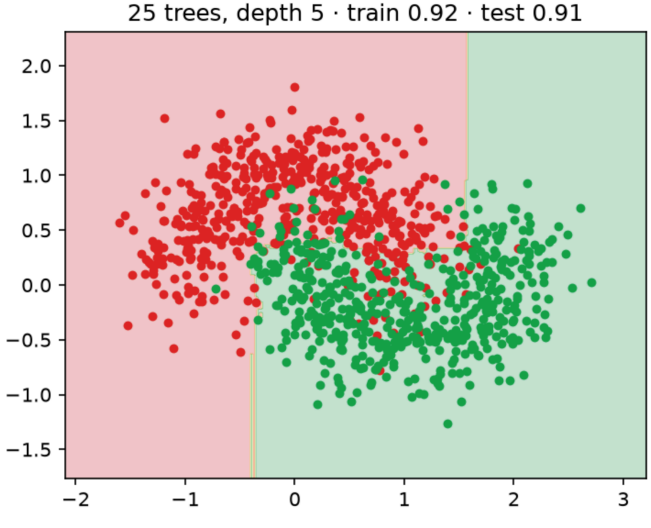
1. **Bagging** — each tree is trained on a random *bootstrap* sample of the data (sampled with replacement; see the right-hand figure)
2. **Random feature subsets** — at each split, only a random subset of features may be considered → the trees become *different*
3. **Aggregate** — majority vote (classification) or average (regression)

Averaging many decorrelated trees cancels their individual errors — the forest is much more robust than any single tree.



n_estimators 25

max_depth per tree 5





Boosting — the other way to combine trees

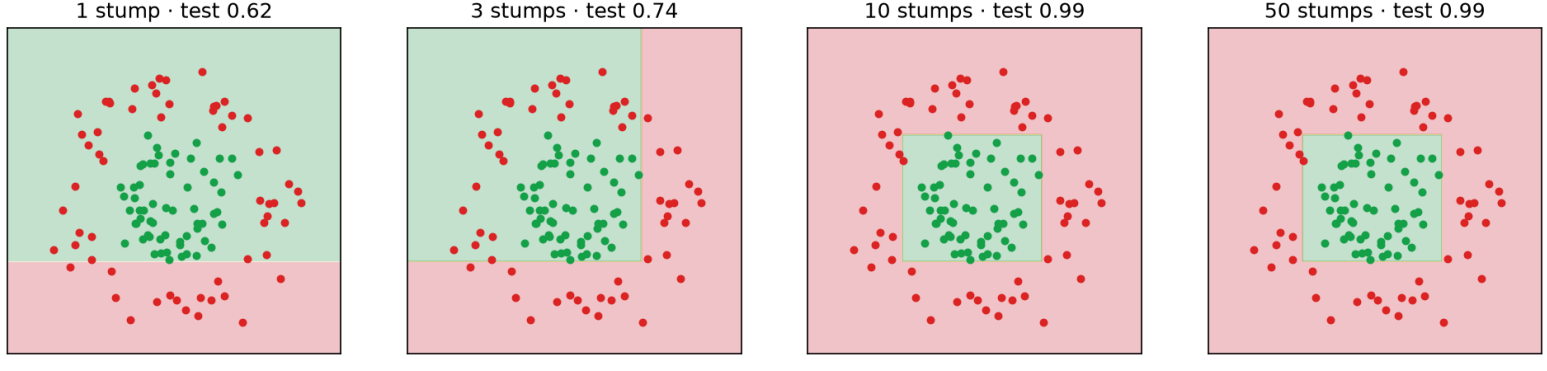
Bagging trains trees **in parallel**; **boosting** trains them **in sequence**, each fixing the previous one's mistakes:

1. **Start simple** — train a weak model (a small tree)
2. **Focus on mistakes** — increase the weight of misclassified samples
3. **Train the next model** — fit another weak learner on the hard cases
4. **Repeat** — each learner corrects its predecessors
5. **Combine** — weighted vote (classification) / weighted sum (regression)

XGBoost (eXtreme Gradient Boosting) is the polished, industrial version of this idea — possibly the most famous ML library of all time, and a Kaggle legend. Trees/boosting dominate **tabular** data leaderboards.

Boosting in action — stumps that fix each other

Each stage is a single **decision stump** (one axis-aligned cut) trained to fix what the previous stages got wrong. A single stump is nearly useless on this ring; a few get it partly right; by the time we have tens of stumps the combined boundary has bent into a **circle** — a shape no single stump could ever produce. Shown on held-out points.



Applications

Trees and ensembles are the workhorses of **tabular** machine learning — most real-world data lives in tables (Excel sheets).

Two Kaggle examples:

- **Otto recommender system** — predict e-commerce clicks, cart additions and orders. *Winning solution:* a 3-layer weighted ensemble of XGBoost, AdaBoost and a neural network.
- **Santander customer satisfaction** — anonymized features. *Winning solution:* a 4-layer ensemble of XGBoost, AdaBoost, random forest and a neural network.

Most data scientists reach for trees, boosting, ensembles and forests — often they outperform neural networks on tabular problems.

Why trees love tables — and not images

A tree asks *"is this one feature above or below a threshold?"*. That is a **meaningful question only when a single feature is informative on its own** — exactly what a well-designed table gives you.

- **Tabular data** — every column is *engineered*: it has a name, a unit and a meaning (age, income, a sensor reading). One cut on a meaningful feature can really separate the classes. This is why trees — and the forests/boosts built on them — **dominate tabular leaderboards**.
- **Images, audio, text** — the raw input is **pixels** (or samples, or token ids), and one pixel value means almost nothing by itself. The information is in *patterns* — edges, textures, shapes and how they sit next to each other. "Pixel $(12, 30) > 0.5$?" is not a meaningful question: shift the object by one pixel and the rule is worthless.

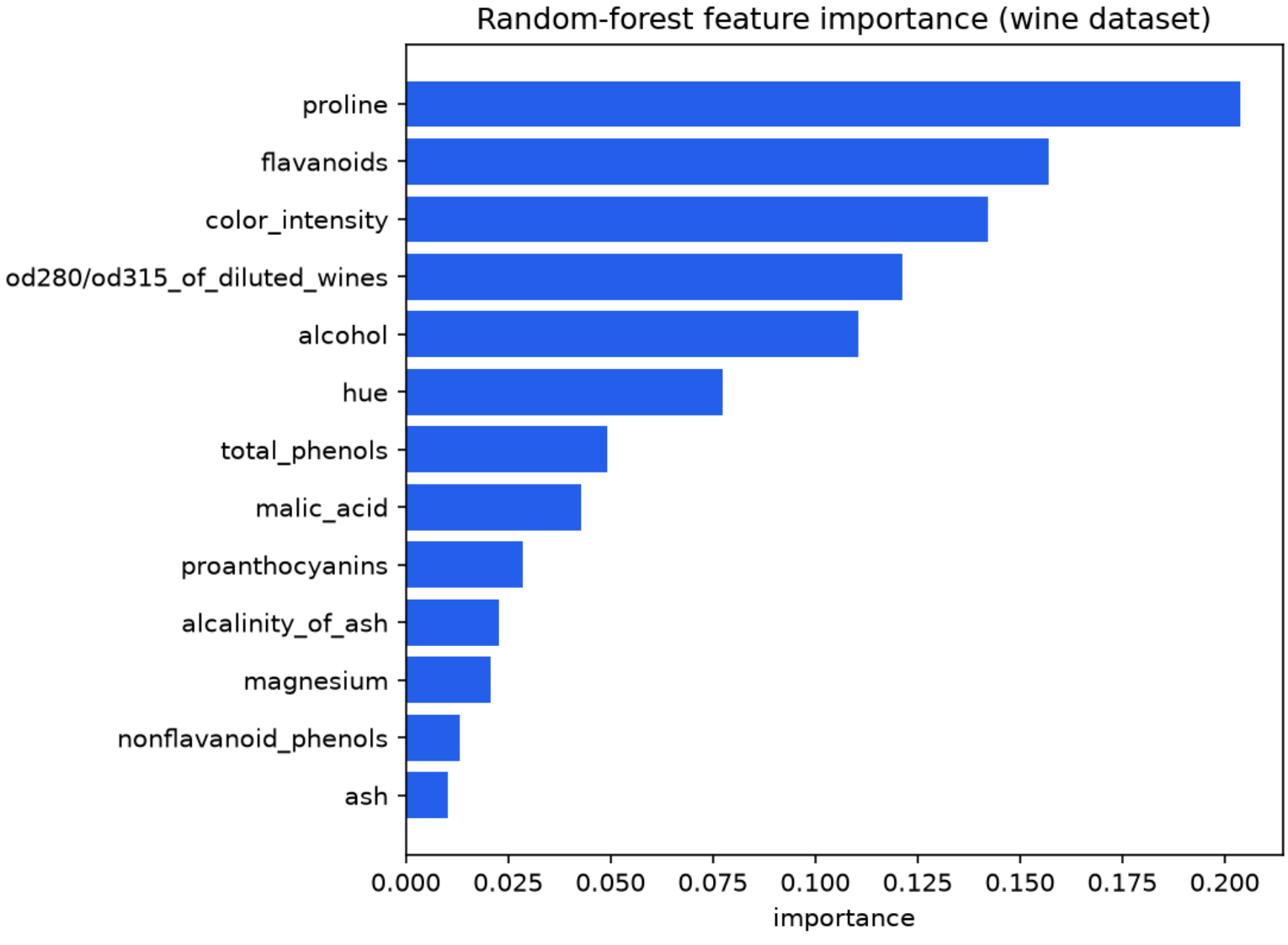
Why the difference? A tree can only make **axis-aligned cuts on one feature at a time**; it cannot *combine* pixels into an edge detector. Recovering a pattern would need an enormous number of nested cuts (a deep, overfitting tree). A **neural network** instead *learns* to combine pixels into features (edges → parts → objects) — which is why **convolutional networks** get the vision jobs and trees get the fraud-detection and forecasting jobs.

What did the forest learn? Feature importance

scikit-learn measures how much each feature **decreased impurity**, averaged over all the trees — a quick and useful ranking of which features matter.

- **Impurity importance** (the default) can be misleading when features have different types or are correlated with each other.
- **Permutation importance** is more reliable: shuffle the values of one feature and see how far the accuracy drops. If shuffling barely changes anything, that feature was not pulling its weight (`sklearn.inspection.permutation_importance`).

In plain words: ask each column *"how much would the forest miss you if we scrambled you?"*


Random-forest feature importance (wine dataset)
proline
flavanoids
color_intensity
od280/od315_of_diluted_wines
alcohol
hue
total_phenols
malic_acid
proanthocyanins
alcalinity_of_ash
magnesium
nonflavanoid_phenols
ash
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
importance

Summary

- **Random forests** average many decorrelated trees (bagging + random feature subsets) — far more robust than a single tree.
- **Boosting** trains trees sequentially, each fixing the last one's mistakes; **XGBoost** is the industrial standard.
- Forests dominate **tabular** machine learning, and their **feature importance** is a useful (if rough) diagnostic.